

```
<pre>
BIP: 173
Layer: Applications
Title: Base32 address format for native v0-16 witness outputs
Authors: Pieter Wuille <pieter.wuille@gmail.com>
        Greg Maxwell <greg@xiph.org>
Comments-Summary: No comments yet.
Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0173
Status: Deployed
Type: Informational
Assigned: 2017-03-20
License: BSD-2-Clause
Replaces: 142
Proposed-Replacement: 350
</pre>
```

==Introduction==

===Abstract===

This document proposes a checksummed base32 format, "Bech32", and a standard for native segregated witness output addresses using it.

===Copyright===

This BIP is licensed under the 2-clause BSD license.

===Motivation===

For most of its history, Bitcoin has relied on base58 addresses with a truncated double-SHA256 checksum. They were part of the original software and their scope was extended in [https://github.com/bitcoin/bips/blob/master/bip-0013.mediawiki BIP13] for Pay-to-script-hash ([https://github.com/bitcoin/bips/blob/master/bip-0016.mediawiki P2SH]). However, both the character set and the checksum algorithm have limitations:

- * Base58 needs a lot of space in QR codes, as it cannot use the 'alphanumeric mode'.
- * The mixed case in base58 makes it inconvenient to reliably write down, type on mobile keyboards, or read out loud.
- * The double SHA256 checksum is slow and has no error-detection guarantees.
- * Most of the research on error-detecting codes only applies to character-set sizes that are a [https://en.wikipedia.org/wiki/Prime_power prime power], which 58 is not.
- * Base58 decoding is complicated and relatively slow.

Included in the Segregated Witness proposal are a new class of outputs (witness programs, see [https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki BIP141]), and two instances of it ("P2WPKH" and "P2WSH", see [https://github.com/bitcoin/bips/blob/master/bip-0143.mediawiki BIP143]). Their functionality is available indirectly to older clients by embedding in P2SH outputs, but for optimal efficiency and security it is best to use it directly. In this document we propose a new address format for native witness outputs (current and future versions).

This replaces [https://github.com/bitcoin/bips/blob/master/bip-0142.mediawiki BIP142], and was previously discussed [https://bitcoincore.org/logs/2016-05-zurich-meeting-notes.html#base32 here] (summarized

[<https://bitcoincore.org/en/meetings/2016/05/20/#error-correcting-codes-for-future-address-types> here]]).

===Examples===

All examples use public key

<tt>0279BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798</tt>.

The P2WSH examples use <tt>key OP_CHECKSIG</tt> as script.

* Mainnet P2WPKH: <tt>bc1qw508d6qejxtdg4y5r3zarvary0c5xw7kv8f3t4</tt>

* Testnet P2WPKH: <tt>tb1qw508d6qejxtdg4y5r3zarvary0c5xw7kxpjzsx</tt>

* Mainnet P2WSH: <tt>bc1qrp33g0q5c5txsp9arysrx4k6zdkfs4nce4xj0gdcccefvpysxf3qccf

mv3</tt>

* Testnet P2WSH: <tt>tb1qrp33g0q5c5txsp9arysrx4k6zdkfs4nce4xj0gdcccefvpysxf3q0sl

5k7</tt>

==Specification==

We first describe the general checksummed base32<ref>'''Why use base32 at all?''

' The lack of mixed case makes it more

efficient to read out loud or to put into QR codes. It does come with a 15% length

increase, but that does not matter when copy-pasting addresses.</ref> format called

'Bech32'' and then define Segregated Witness addresses using it.

===Bech32===

A Bech32<ref>'''Why call it Bech32?'''' "Bech" contains the characters BCH (the error

detection algorithm used) and sounds a bit like "base".</ref> string is at most 90 characters long and consists of:

* The ''human-readable part'', which is intended to convey the type of data, or anything else that is relevant to the reader. This part MUST contain 1 to 83 U S-ASCII characters, with each character having a value in the range [33-126]. HR P validity may be further restricted by specific applications.

* The ''separator'', which is always "1". In case "1" is allowed inside the human-readable part, the last one in the string is the separator<ref>'''Why include a separator in addresses?'''

That way the human-readable part is unambiguously separated from the data part, avoiding potential collisions with other human-readable parts that share a prefix. It also allows us to avoid having character-set restrictions on the human-readable part.

The

separator is ''1'' because using a non-alphanumeric character would complicate copy-pasting of addresses (with no double-click selection in several applications). Therefore an alphanumeric character outside the normal character set

was chosen.</ref>.

* The ''data part'', which is at least 6 characters long and only consists of alphanumeric characters excluding "1", "b", "i", and "o"<ref>'''Why not use an existing character set like [<http://www.faqs.org/rfcs/rfc3548.html> RFC3548] or [<https://philzimmermann.com/docs/human-oriented-base-32-encoding.txt> z-base-32]''

?

The character set is chosen to minimize ambiguity according to [<https://hissa.nist.gov/~black/GTLD/> this] visual similarity data, and the ordering is chosen to minimize the number of pairs of similar characters (according to the same data) that differ in more than 1 bit.

As the checksum is chosen to maximize detection capabilities for low numbers of bit errors, this choice improves its performance under some error models.</ref>.

```

{| class="wikitable"
|-
|0
|1
|2
|3
|4
|5
|6
|7
|-
|+0
|q||p||z||r||y||9||x||8
|-
|+8
|g||f||2||t||v||d||w||0
|-
|+16
|s||3||j||n||5||4||k||h
|-
|+24
|c||e||6||m||u||a||7||l
|}

```

'''Checksum'''

The last six characters of the data part form a checksum and contain no information. Valid strings **MUST** pass the criteria for validity specified by the Python3 code snippet below. The function `bech32_verify_checksum` must return true when its arguments are:

- * `hrp`: the human-readable part as a string
- * `data`: the data part as a list of integers representing the characters after conversion using the table above

```

<pre>
def bech32_polymod(values):
    GEN = [0x3b6a57b2, 0x26508e6d, 0x1ea119fa, 0x3d4233dd, 0x2a1462b3]
    chk = 1
    for v in values:
        b = (chk >> 25)
        chk = (chk & 0xffffffff) << 5 ^ v
        for i in range(5):
            chk ^= GEN[i] if ((b >> i) & 1) else 0
    return chk

def bech32_hrp_expand(s):
    return [ord(x) >> 5 for x in s] + [0] + [ord(x) & 31 for x in s]

def bech32_verify_checksum(hrp, data):
    return bech32_polymod(bech32_hrp_expand(hrp) + data) == 1
</pre>

```

This implements a [https://en.wikipedia.org/wiki/BCH_code BCH code] that guarantees detection of '''any error affecting at most 4 characters''' and has less than a 10^{-9} chance of failing to detect more errors. More details about the properties can be found in the Checksum Design appendix. The human-readable part is processed by first feeding the higher bits of each character's US-ASCII value into the checksum calculation followed by a zero and then the lower bits of each<ref>'''Why are the high bits of the human-readable part processed first?'''

This results in the actually checksummed data being `''[high hrp] 0 [low hrp] [data]''`. This means that under the assumption that errors to the human readable part only change the low 5 bits (like changing an alphabetical character into another), errors are restricted to the `''[low hrp] [data]''` part, which is at most 89 characters, and thus all error detection properties (see appendix) remain applicable.</ref>.

To construct a valid checksum given the human-readable part and (non-checksum) values of the data-part characters, the code below can be used:

```
<pre>def bech32_create_checksum(hrp, data):
    values = bech32_hrp_expand(hrp) + data
    polymod = bech32_polymod(values + [0,0,0,0,0,0]) ^ 1
    return [(polymod >> 5 * (5 - i)) & 31 for i in range(6)]
</pre>
```

`'''Error correction'''`

One of the properties of these BCH codes is that they can be used for error correction. An unfortunate side effect of error correction is that it erodes error detection: correction changes invalid inputs into valid inputs, but if more than a few errors were made then the valid input may not be the correct input. Use of an incorrect but valid input can cause funds to be lost irrecoverably. Because of this, implementations SHOULD NOT implement correction beyond potentially suggesting to the user where in the string an error might be found, without suggesting the correction to make.

`'''Uppercase/lowercase'''`

The lowercase form is used when determining a character's value for checksum purposes.

Encoders MUST always output an all lowercase Bech32 string. If an uppercase version of the encoding result is desired, (e.g.- for presentation purposes, or QR code use), then an uppercasing procedure can be performed external to the encoding process.

Decoders MUST NOT accept strings where some characters are uppercase and some are lowercase (such strings are referred to as mixed case strings).

For presentation, lowercase is usually preferable, but inside QR codes uppercase SHOULD be used, as those permit the use of `''[http://www.thonky.com/qr-code-tutorial/alphanumeric-mode-encoding alphanumeric mode]''`, which is 45% more compact than the normal `''[http://www.thonky.com/qr-code-tutorial/byte-mode-encoding byte mode]''`.

`===Segwit address format===`

A segwit address<ref>'''Why not make an address format that is generic for all scriptPubKeys?'''

That would lead to confusion about addresses for existing scriptPubKey types. Furthermore, if addresses that do not have a one-to-one mapping with scriptPubKeys (such as ECDH-based addresses) are ever introduced, having a fully generic old address type available would permit reinterpreting the resulting scriptPubKeys using the old address format, with lost funds as a result if bitcoins are sent to them.</ref> is a Bech32 encoding of:

* The human-readable part "bc"<ref>'''Why use 'bc' as human-readable part and no

t 'btc'?''' 'bc' is shorter.</ref> for mainnet, and "tb"<ref>'''Why use 'tb' as human-readable part for testnet?''' It was chosen to be of the same length as the mainnet counterpart (to simplify implementations' assumptions about lengths), but still be visually distinct.</ref> for testnet.

* The data-part values:

** 1 character (representing 5 bits of data): the witness version

** A conversion of the 2-to-40-byte witness program (as defined by [https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki BIP141]) to base32:

*** Start with the bits of the witness program, most significant bit per byte first.

*** Re-arrange those bits into groups of 5, and pad with zeroes at the end if needed.

*** Translate those bits to characters using the table above.

'''Decoding'''

Software interpreting a segwit address:

* MUST verify that the human-readable part is "bc" for mainnet and "tb" for testnet.

* MUST verify that the first decoded data value (the witness version) is between 0 and 16, inclusive.

* Convert the rest of the data to bytes:

** Translate the values to 5 bits, most significant bit first.

** Re-arrange those bits into groups of 8 bits. Any incomplete group at the end MUST be 4 bits or less, MUST be all zeroes, and is discarded.

** There MUST be between 2 and 40 groups, which are interpreted as the bytes of the witness program.

Decoders SHOULD enforce known-length restrictions on witness programs.

For example, BIP141 specifies 'If the version byte is 0, but the witness program is neither 20 nor 32 bytes, the script must fail.'

As a result of the previous rules, addresses are always between 14 and 74 characters long, and their length modulo 8 cannot be 0, 3, or 5.

Version 0 witness addresses are always 42 or 62 characters, but implementations MUST allow the use of any version.

Implementations should take special care when converting the address to a scriptPubkey, where witness version 'n' is stored as 'OP_n'. OP_0 is encoded as 0x00, but OP_1 through OP_16 are encoded as 0x51 through 0x60 (81 to 96 in decimal). If a bech32 address is converted to an incorrect scriptPubKey the result will likely be either unspendable or insecure.

===Compatibility===

Only new software will be able to use these addresses, and only for receivers with segwit-enabled new software. In all other cases, P2SH or P2PKH addresses can be used.

==Rationale==

<references />

==Reference implementations==

* Reference encoder and decoder:

** [https://github.com/sipa/bech32/tree/master/ref/c For C]

** [https://github.com/sipa/bech32/tree/master/ref/c++ For C++]

** [https://github.com/sipa/bech32/tree/master/ref/javascript For JavaScript]

** [https://github.com/sipa/bech32/tree/master/ref/go For Go]

** [https://github.com/sipa/bech32/tree/master/ref/python For Python]


```
* <tt>bc1zw508d6qejxtdg4y5r3zarvaryvg6kdaj</tt>: <tt>5210751e76e8199196d454941c4
5d1b3a323</tt>
* <tt>tb1qqqqqp399et2xygdj5xreqhjvcmzhxw4aywxecjdzew6hylvgsesrxh6hy</tt>: <tt>0
020000000c4a5cad46221b2a187905e5266362b99d5e91c6ce24d165dab93e86433</tt>
```

The following list gives invalid segwit addresses and the reason for their invalidity.

```
* <tt>tc1qw508d6qejxtdg4y5r3zarvary0c5xw7kg3g4ty</tt>: Invalid human-readable part
* <tt>bc1qw508d6qejxtdg4y5r3zarvary0c5xw7kv8f3t5</tt>: Invalid checksum
* <tt>BC13W508D6QEJXTDG4Y5R3ZARVARY0C5XW7KN40WF2</tt>: Invalid witness version
* <tt>bc1rw5uspcuh</tt>: Invalid program length
* <tt>bc10w508d6qejxtdg4y5r3zarvary0c5xw7kw508d6qejxtdg4y5r3zarvary0c5xw7kw5rljs
90</tt>: Invalid program length
* <tt>BC1QR508D6QEJXTDG4Y5R3ZARVARYV98GJ9P</tt>: Invalid program length for witness version 0 (per BIP141)
* <tt>tb1qrp33g0q5c5txsp9arysrx4k6zdkfs4nce4xj0gdcccefvpysxf3q0sL5k7</tt>: Mixed case
* <tt>bc1zw508d6qejxtdg4y5r3zarvaryvqyzf3du</tt>: zero padding of more than 4 bits
* <tt>tb1qrp33g0q5c5txsp9arysrx4k6zdkfs4nce4xj0gdcccefvpysxf3pjxtptv</tt>: Non-zero padding in 8-to-5 conversion
* <tt>bc1gmk9yu</tt>: Empty data section
```

===Checksum design===

'''Design choices'''

BCH codes can be constructed over any prime-power alphabet and can be chosen to have a good trade-off between size and error-detection capabilities. While most work around BCH codes uses a binary alphabet, that is not a requirement. This makes them more appropriate for our use case than [https://en.wikipedia.org/wiki/Cyclic_redundancy_check CRC codes]. Unlike [[https://en.wikipedia.org/wiki/Reed-Solomon codes](https://en.wikipedia.org/wiki/Reed%E2%80%93Solomon_error_correction)], they are not restricted in length to one less than the alphabet size. While they also support efficient error correction, the implementation of just error detection is very simple.

We pick 6 checksum characters as a trade-off between length of the addresses and the error-detection capabilities, as 6 characters is the lowest number sufficient for a random failure chance below 1 per billion. For the length of data we're interested in protecting (up to 71 bytes for a potential future 40-byte witness program), BCH codes can be constructed that guarantee detecting up to 4 errors.

'''Selected properties'''

Many of these codes perform badly when dealing with more errors than they are designed to detect, but not all. For that reason, we consider codes that are designed to detect only 3 errors as well as 4 errors, and analyse how well they perform in practice.

The specific code chosen here is the result of:

```
* Starting with an exhaustive list of 159605 BCH codes designed to detect 3 or 4 errors up to length 93, 151, 165, 341, 1023, and 1057.
* From those, requiring the detection of 4 errors up to length 71, resulting in 28825 remaining codes.
```

- * From those, choosing the codes with the best worst-case window for 5-character errors, resulting in 310 remaining codes.
- * From those, picking the code with the lowest chance for not detecting small numbers of 'bit' errors.

As a naive search would require over $6.5 * 10^{19}$ checksum evaluations, a collision-search approach was used for analysis. The code can be found [<https://github.com/sipa/ezbase32/> here].

'''Properties'''

The following table summarizes the chances for detection failure (as multiples of 1 in 10^9).

Length		Description	
≤4			
5			
6			
7			
8			
≥9			
8		Longest detecting 6 errors	colspan="3" 0 1.127 0.909 n/a
18		Longest detecting 5 errors	colspan="2" 0 0.965 0.929 0.932 0.931
19		Worst case for 6 errors	0 0.093 0.972 0.928 colspan="2" 0.931
39		Length for a P2WPKH address	0 0.756 0.935 0.932 colspan="2" 0.931
59		Length for a P2WSH address	0 0.805 0.933 colspan="3" 0.931
71		Length for a 40-byte program address	0 0.830 0.934 colspan="3" 0.931
89		Longest detecting 4 errors	0 0.867 0.933 colspan="3" 0.931

This means that when 5 changed characters occur randomly distributed in the 39 characters of a P2WPKH address, there is a chance of '0.756 per billion' that it will go undetected. When those 5 changes occur randomly within a 19-character window, that chance goes down to '0.093 per billion'. As the number of errors goes up, the chance converges towards '1 in 2^{30} ' = '0.931 per billion'.

Even though the chosen code performs reasonably well up to 1023 characters, other designs are preferable for lengths above 89 characters (excluding the separator).

==Acknowledgements==

This document is inspired by the [<https://rusty.ozlabs.org/?p=578> address proposal] by Rusty Russell, the [<https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2014-February/004402.htm>]

ml base32] proposal by Mark Friedenbach, and had input from Luke Dashjr, Johnson Lau, Eric Lombrozo, Peter Todd, and various other reviewers.

==Disclosures (added 2024)==

Due to an oversight in the design of bech32, this checksum scheme is not always robust against

[[<https://gist.github.com/sipa/a9845b37c1b298a7301c33a04090b2eb>|the insertion and deletion of fewer than 5 consecutive characters]]. Due to this weakness, [[bip-0350.mediawiki|BIP-350]] proposes using the scheme described in this BIP only for Native Segwit v0 outputs.